QCV: Quantum Circuit Code Generation using Visual Capabilities of Multi-Modal Large Language Models

Dongping Liu
Tenorshare (Hong Kong) Limited
Hong Kong, China
steven.dpliu@gmail.com

Aoyu Zhang
Amazon Web Services
Beijing, China
aoyuzhan@amazon.com

Luyao Zhang
Duke Kunshan University
Suzhou, China
lz183@duke.edu

***Abstract*—Quantum circuit diagrams are the primary visual representation in quantum computing research, yet translating them into executable code remains a manual and error-prone process. We present QCV-Dataset, the first multimodal quantum circuit dataset comprising 132 circuits across 13 categories (1–10 qubits), with seven modalities per circuit: diagram images, executable code in two SDKs (Amazon Braket and Qiskit), simulation results, bilingual structured annotations, natural language target descriptions, and optimization equivalence pairs—plus 27 annotated failure cases. Using this dataset, we evaluate three multimodal large language models (MMLLMs) under two visual modes: Basic Vision (BV) and Thinking Vision (TV), a chain-of-thought prompting method adapted from our prior work on classical circuit diagrams. The strongest model achieves a 97.0% weighted score under BV on 21 evaluated circuits, demonstrating that MMLLMs can reliably interpret quantum circuit diagrams. We discover a *CoT sensitivity window*: chain-of-thought reasoning only impacts performance near a model's capability boundary—improving weaker models (up to +20pp) on simple tasks but degrading performance on advanced circuits (-17pp), while leaving the mid-tier model entirely unaffected. All generated code is verified end-to-end using unitary matrix fidelity on a local quantum simulator. The dataset includes 31 blockchain-relevant circuits covering quantum threats to Bitcoin (Shor vs. ECDSA, Grover vs. SHA-256), post-quantum defenses (Kyber, Dilithium, SPHINCS+), and quantum-enhanced infrastructure (QKD networks, random beacons). Dataset and code are publicly available at https://github.com/QuantBlockchain/quantum-circuit-vision.**

I. Introduction

Quantum computing has advanced rapidly in recent years, with quantum algorithms demonstrating theoretical advantages in optimization [7], simulation [8], and cryptography [9]. However, a persistent gap exists between algorithmic descriptions in research papers and their implementation as executable code. Quantum circuit diagrams—the standard visual notation for quantum algorithms—appear ubiquitously in textbooks and publications, yet translating these diagrams into runnable code on platforms such as Amazon Braket [10], Qiskit [11], or Cirq requires manual effort and domain expertise. This translation is tedious and error-prone, particularly for circuits involving multi-qubit entangling gates and parameterized rotations. The need for rapid prototyping is especially acute in emerging domains such as post-quantum cryptography (PQC), where quantum circuits are used to analyze the security of blockchain systems [16] and design quantum-safe protocols [17].

Recent work has demonstrated that large language models (LLMs) can assist in hardware design automation. Projects such as ChipChat [2], VeriGen [3], GPT4AIGChip [4], ChipNemo [5], and ChatEDA [6] have explored using LLMs to generate HDL code, EDA scripts, and accelerator designs from textual specifications. Multimodal models such as GPT-4 [15] and LLaVA [14] have further demonstrated strong visual reasoning capabilities across domains including mathematical problem solving [12]. Our prior work, VGV [1], extended this line of research to the visual domain, showing that MMLLMs can generate Verilog code from classical circuit diagrams. VGV introduced the Thinking Vision (TV) prompting method, which improved open-source model performance by over 50% compared to Basic Vision (BV).

Quantum circuit diagrams possess several properties that make them particularly amenable to visual code generation by MMLLMs. First, quantum gate symbols follow standardized conventions (e.g., H for Hadamard, $\oplus$ for CNOT targets, $\bullet$ for control qubits), unlike classical microarchitecture diagrams which lack industry-wide standardization. Second, quantum circuits have a strictly left-to-right temporal structure with no feedback loops. Third, the gate vocabulary is finite and well-defined. These properties suggest that MMLLMs may achieve higher accuracy on quantum circuits than on classical ones.

In this paper, we present QCV (Quantum Circuit Vision), the first systematic study of MMLLM-based code generation from quantum circuit diagram images, accompanied by QCV-Dataset, the first multimodal quantum circuit dataset. Our contributions are:

1) We construct QCV-Dataset: 132 quantum circuits across 13 categories (1–10 qubits) with seven modalities per circuit—diagram images, executable code in two SDKs (Braket and Qiskit), simulation results, bilingual annotations, natural language target descriptions, and optimization equivalence pairs—publicly released for community use.
2) We evaluate three MMLLMs (Claude Opus 4.6, Sonnet 4.6, and Haiku 4.5) under BV and TV visual modes on

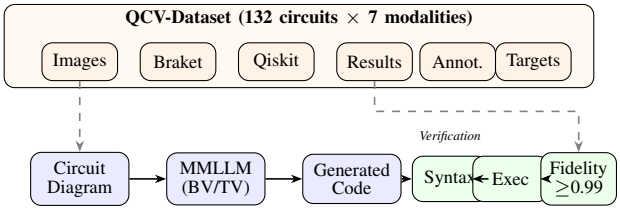


Fig. 1. QCV system overview. Top: QCV-Dataset provides 132 circuits with 7 modalities (images, dual-SDK code, simulation results, annotations, target descriptions, equivalence pairs). Bottom: evaluation pipeline processes circuit images through MMLLMs and verifies generated code via syntax checking, execution, and unitary fidelity.

21 circuits, discovering a *CoT sensitivity window*: TV's effect depends on model capability level, a finding not observed in prior classical circuit experiments.

3) We propose an end-to-end verification pipeline using unitary matrix fidelity on Amazon Braket's LocalSimulator, and validate on 31 blockchain-relevant circuits covering quantum threats, post-quantum defenses, and quantum infrastructure.
4) We release all code, data, and annotations at https://github.com/QuantBlockchain/quantum-circuit-vision to enable reproducibility and future research toward AI-driven quantum circuit discovery.

Our results show that Claude Opus 4.6 under BV achieves a weighted score of 97.0%, passing 20 out of 21 circuits, Sonnet 4.6 scores 88.5%, and Haiku 4.5 scores 46.5%. The performance gap between models widens with circuit complexity (40pp between Opus and Haiku at Basic, 67pp at Advanced), indicating that advanced visual reasoning capabilities are critical for complex quantum circuit understanding.

II. Methodology

Following the approach established in VGV [1], we design a difficulty-graded benchmark, define two visual prompting modes, and build an end-to-end verification pipeline tailored to quantum circuits.

A. Benchmark Design

We construct QCV-Dataset, a multimodal quantum circuit dataset of 132 circuits across 13 categories, with seven data modalities per circuit. For the evaluation reported in this paper, we select a representative subset of 21 circuits across three difficulty levels, inspired by standard quantum computing curricula [8]. All circuit diagrams are generated programmatically using Qiskit's `matplotlib` drawer [11], ensuring reproducible and unambiguous visual inputs. Ground truth implementations are provided in both Amazon Braket SDK [10] and Qiskit.

The full dataset spans 13 categories: basic gates (5), intermediate algorithms (10), advanced algorithms (6), blockchain protocols (11), gate-type coverage (15), qubit scaling 4–10q (12), classical algorithms (15), variational circuits (10), error correction codes (8), quantum ML (10), blockchain extended (8), visual variants (10), and BTC/blockchain quantum security (12). The 21 evaluated circuits are drawn from the first three categories:

- **Basic (5 circuits):** Single-gate or single-entangling-gate circuits testing fundamental gate recognition (Hadamard, CNOT, Bell state, GHZ state, Toffoli).
- **Intermediate (10 circuits):** Multi-gate circuits requiring understanding of gate composition, control relationships, and parameterized rotations (SWAP decomposition, 2-qubit QFT, teleportation, Deutsch, superdense coding, Grover 2-qubit, parameterized rotation, Fredkin, shift register, phase estimation).
- **Advanced (6 circuits):** Complete quantum algorithms involving complex multi-qubit structures (3-qubit QFT, 3-qubit Grover, VQE ansatz, QAOA, quantum walk, Bernstein-Vazirani).

TABLE I
QCV-Dataset: 132 Circuits across 13 Categories

Category	Focus	Count	Qubits
Basic (demo)	Fundamental gates	5	1–3
Intermediate (inter)	Multi-gate composition	10	2–4
Advanced (adv)	Complete algorithms	6	3–5
Blockchain	Quantum protocols	11	2–8
A: Gate Coverage	All standard gate types	15	1–3
B: Qubit Scaling	4–10 qubit circuits	12	4–10
C: Classical Algorithms	Textbook algorithms	15	2–4
D: Variational	NISQ-era ansätze	10	2–4
E: Error Correction	QEC codes	8	3–9
F: Quantum ML	QNN, QCNN, kernels	10	2–8
G: Blockchain Ext.	Crypto protocols	8	3–6
H: Visual Variants	Layout robustness	10	2–4
I: BTC Security	Threats & PQC defense	12	4–7
Total		**132**	**1–10**

Each circuit in the full dataset includes: (1) a PNG diagram image, (2) executable Braket SDK code, (3) executable Qiskit code, (4) simulation results (state vectors from LocalSimulator), (5) structured bilingual annotations (English/Chinese), (6) a natural language target description for circuit synthesis tasks, and (7) optimization equivalence pairs where applicable. Additionally, 27 annotated failure cases from our MMLLM experiments are included for error analysis research. Table I summarizes the 13 categories, and Table II lists the 21 evaluated circuits.

B. Visual Modes

We evaluate two visual prompting strategies adapted from VGV [1]:

Basic Vision (BV) provides the circuit diagram image with a direct code generation instruction:

> *"Please generate Amazon Braket SDK Python code based on this quantum circuit diagram. Requirements: use* `from braket.circuits import Circuit`*; output only complete executable Python code; no explanation, no markdown."*

Thinking Vision (TV) adds a chain-of-thought analysis step before code generation:

TABLE II
EVALUATED SUBSET: 21 QUANTUM CIRCUITS

No.	Difficulty	Circuit	Qubits
1	Basic	Hadamard	1
2	Basic	CNOT	2
3	Basic	Bell State	2
4	Basic	GHZ State	3
5	Basic	Toffoli (CCX)	3
6	Intermediate	SWAP Decomposition	2
7	Intermediate	2-Qubit QFT	2
8	Intermediate	Teleportation Prep	3
9	Intermediate	Deutsch Algorithm	2
10	Intermediate	Superdense Coding	2
11	Intermediate	2-Qubit Grover	2
12	Intermediate	Parameterized Rotation	2
13	Intermediate	Fredkin (CSWAP)	3
14	Intermediate	Shift Register	4
15	Intermediate	Phase Estimation	3
16	Advanced	3-Qubit QFT	3
17	Advanced	3-Qubit Grover	3
18	Advanced	VQE Ansatz	4
19	Advanced	QAOA (MaxCut)	4
20	Advanced	Quantum Walk	4
21	Advanced	Bernstein-Vazirani	5

"Please first analyze this quantum circuit diagram: 1. How many qubit lines? What are their labels? 2. What gates appear from left to right? 3. Which gates have control qubits? Which line is control, which is target? Then generate Amazon Braket SDK Python code based on your analysis."

Unlike VGV, which also varied prompt detail levels (Short-/Medium/High), we use a single prompt per mode to isolate the effect of chain-of-thought reasoning from prompt informativeness.

C. Verification Pipeline

Generated code is verified through a three-level pipeline:

1) **Syntax check:** The extracted Python code is compiled using `compile()` to detect syntax errors.
2) **Execution check:** The code is executed in a sandboxed namespace. We verify that a valid Braket `Circuit` object is produced.
3) **Unitary fidelity check:** We compute the full unitary matrix of both the generated circuit and the ground truth by running each computational basis state on Amazon Braket's LocalSimulator [10] with `shots=0`. The fidelity is computed as $F = |\mathrm{Tr}(U_{\mathrm{gt}}^{\dagger} U_{\mathrm{gen}})|/d$, where $d = 2^n$ is the Hilbert space dimension. A circuit passes if $F \geq 0.99$.

This unitary-based verification is more rigorous than state-vector comparison on a single input, as it checks correctness across all possible input states. It is also phase-tolerant, correctly handling circuits that differ only by a global phase.

D. Evaluation Metrics

We report two metrics following VGV [1]:

TABLE III
EVALUATED MMLLMS

Model	Tier	Relative Cost
Claude Opus 4.6	High-end	2.20×
Claude Sonnet 4.6	Mid-tier	1.30×
Claude Haiku 4.5	Lightweight	0.40×

TABLE IV
PASS RATES BY MODEL, VISUAL MODE, AND DIFFICULTY

		Pass Rate			Wtd.
Model	Mode	Bas.	Int.	Adv.	Score
Opus	BV	5/5 (100%)	9/10 (90%)	6/6 (100%)	**97.0%**
	TV	5/5 (100%)	10/10 (100%)	5/6 (83%)	91.5%
Sonnet	BV	5/5 (100%)	9/10 (90%)	5/6 (83%)	88.5%
	TV	5/5 (100%)	9/10 (90%)	5/6 (83%)	88.5%
Haiku	BV	3/5 (60%)	6/10 (60%)	2/6 (33%)	46.5%
	TV	4/5 (80%)	7/10 (70%)	1/6 (16%)	45.0%

Pass rate is the fraction of circuits for which the generated code passes all three verification levels, reported per difficulty level.

Weighted score aggregates pass rates across difficulty levels: $S = 0.2 \times R_{\mathrm{Basic}} + 0.3 \times R_{\mathrm{Intermediate}} + 0.5 \times R_{\mathrm{Advanced}}$, where R denotes the pass rate at each level. The weights reflect increasing difficulty, with Advanced circuits contributing the most to the overall score.

III. EVALUATION

A. Experimental Setup

We evaluate three MMLLMs from the Claude family, spanning high-end, mid-tier, and lightweight capability levels (Table III). All models are accessed through a unified command-line interface (`kiro-cli`) in non-interactive mode, ensuring identical execution conditions. Each circuit diagram is provided as a PNG image alongside the BV or TV prompt. The total experiment comprises 21 circuits × 3 models × 2 visual modes = 126 invocations.

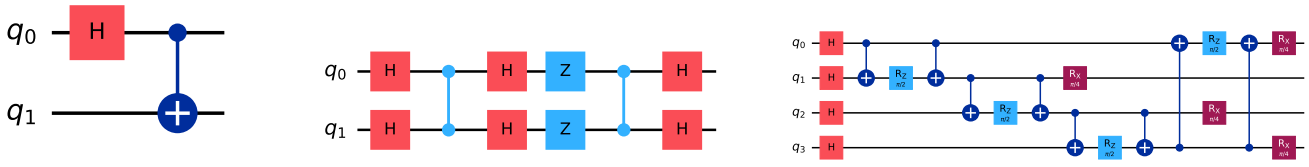
All generated code is verified on Amazon Braket's LocalSimulator (a free, local state-vector simulator) using the unitary fidelity pipeline described in Section II-C. No cloud quantum hardware is used.

B. Results

Table IV presents the complete pass rate matrix across models, visual modes, and difficulty levels.

Opus 4.6 under BV achieves the highest weighted score (97.0%), passing 20 of 21 circuits. Its sole failure is an API naming error (`.crz()` instead of `.cphaseshift()`) on the phase estimation circuit. Under TV, Opus achieves a perfect 10/10 on Intermediate but drops to 5/6 on Advanced due to an incorrect QAOA implementation (fidelity = 0.25).

Sonnet 4.6 scores 88.5% under both BV and TV—identical results across all difficulty levels. It fails on phase estimation (BV: structural error with fidelity 0.23; TV: API error using



(a) Basic: Bell State (2 qubits, 2 gates) (b) Intermediate: 2-Qubit Grover (2 qubits, 9 gates) (c) Advanced: QAOA MaxCut (4 qubits, 16 gates)

Fig. 2. Representative circuit diagrams from each difficulty level, generated using Qiskit's matplotlib drawer. Complexity increases from left to right in qubit count, gate count, and structural depth.

`.crz()`) and QAOA (BV: fidelity 0.50; TV: fidelity 0.15) under both modes, though with different failure mechanisms.

Haiku 4.5 scores substantially lower across all conditions. Under BV, it passes 11 of 21 circuits; under TV, 12 of 21. Its weighted scores (46.5% BV, 45.0% TV) are roughly half of Opus's, with the gap widening at higher difficulty levels. Note that the weighted scores differ substantially from raw pass rates (e.g., 11/21 = 52% raw vs. 46.5% weighted) because Advanced circuits carry 50% of the weight.

IV. Analysis

A. Visual Modes: CoT Sensitivity Window

The most notable finding is that TV's effect depends not only on circuit complexity but also on model capability level (Fig. 3):

Haiku (weak): TV has a strong positive effect on simpler circuits that diminishes and reverses with complexity. Basic: +20pp (TV helps select correct API, e.g., `.ccnot()` instead of `.toffoli()`). Intermediate: +10pp. Advanced: −17pp.

Opus (strong): TV has a moderate positive effect on intermediate circuits but reverses on advanced ones. Basic: 0pp (already at ceiling). Intermediate: +10pp. Advanced: −17pp.

Sonnet (mid-tier): TV has *zero effect* at all difficulty levels. BV and TV produce identical pass rates (100%/90%/83%). Notably, Sonnet fails on the same circuits (phase estimation, QAOA) under both modes but through different mechanisms—BV produces a structural error (fidelity 0.23) on phase estimation while TV produces an API error (`.crz()`), indicating that TV alters the reasoning pathway without changing the outcome.

These results suggest a *CoT sensitivity window*: chain-of-thought prompting only affects performance when task difficulty is near the model's capability boundary. For Haiku, even Basic circuits are near this boundary, so TV helps across a wide range. For Opus, only Intermediate circuits benefit—Basic is too easy (ceiling effect) and Advanced triggers over-analysis. For Sonnet, the model's capability is sufficiently above Basic/Intermediate and sufficiently below Advanced that TV neither helps nor hurts. The −17pp reversal on Advanced circuits is shared by Opus and Haiku but absent in Sonnet, suggesting it requires either very strong holistic pattern matching (Opus) or very limited capacity that TV further degrades (Haiku).

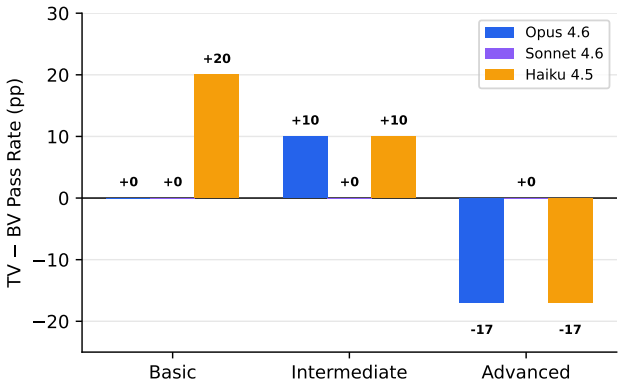


Fig. 3. TV−BV pass rate difference by model and difficulty. Opus and Haiku show a reversal from positive to negative; Sonnet is entirely unaffected.

BV Output (Fail)

```
circuit = Circuit()
circuit.toffoli(
  control_qubits=[0, 1],
  target_qubit=2)
```

`.toffoli()` does not exist in Braket SDK

TV Output (Pass)

```
circuit = Circuit()
circuit.ccnot(0, 1, 2)
```

Correct API: `.ccnot()`, fidelity = 1.0

Fig. 4. Haiku on the Toffoli circuit: BV uses a nonexistent API, while TV's analysis leads to the correct call—an example of CoT helping near the capability boundary.

This pattern was not observed in VGV [1], where TV consistently improved performance. We attribute this to the tightly coupled multi-qubit structures in advanced quantum circuits (e.g., oracle–diffusion pairs in Grover, cost–mixer layers in QAOA), which may be disrupted by step-by-step decomposition in ways that classical circuit structures are not.

B. Model Comparison

Under BV, the three models form a clear capability hierarchy:

- Opus: 97.0% weighted (20/21 pass)
- Sonnet: 88.5% weighted (19/21 pass)
- Haiku: 46.5% weighted (11/21 pass)

Sonnet sits closer to Opus than to Haiku, suggesting that mid-tier models already possess strong quantum circuit visual understanding. The Opus–Haiku gap widens with difficulty (Basic 40pp, Intermediate 30pp, Advanced 67pp), while the

Opus–Sonnet gap is smaller and more uniform (Basic 0pp, Intermediate 0pp, Advanced 17pp).

Opus BV passes 20/21 circuits overall (95.2%), with its single failure being an API naming issue rather than a visual comprehension error. Sonnet shares the same phase estimation and QAOA failures as Opus TV, suggesting these circuits represent genuine difficulty boundaries rather than random errors.

C. Error Analysis

The two models exhibit qualitatively different failure modes. **Opus** failures are almost exclusively API naming errors: using Qiskit-style method names (`.crz()`, `.cx()`) instead of Braket equivalents (`.cphaseshift()`, `.cnot()`). The visual circuit interpretation is correct in all cases—the model identifies the right gates, qubit assignments, and ordering, but maps them to incorrect SDK calls. **Haiku** failures, by contrast, are diverse: API naming errors (`.toffoli()` instead of `.ccnot()`, keyword argument mismatches), visual misidentification of gate types, CNOT control/target reversal, and structural errors such as incorrect gate ordering or missing oracle components in Grover circuits. This diversity suggests that Haiku's limitations lie in fundamental visual comprehension rather than SDK knowledge alone, whereas Opus's errors could likely be eliminated with SDK-specific post-processing.

V. Conclusion

This paper presents QCV-Dataset, the first multimodal quantum circuit dataset, and a systematic evaluation of MMLLMs for generating executable quantum code from circuit diagram images. QCV-Dataset comprises 132 circuits across 13 categories (1–10 qubits) with seven modalities per circuit: diagram images, dual-SDK code (Braket and Qiskit), simulation results, bilingual annotations, natural language target descriptions, and optimization equivalence pairs.

Our evaluation on 21 circuits demonstrates that MMLLMs can reliably interpret quantum circuit diagrams: Claude Opus 4.6 achieves a 97.0% weighted score under BV, failing only on an SDK-specific API naming issue. The most significant finding is a *CoT sensitivity window*—TV's effectiveness depends on the interaction between model capability and task difficulty. TV helps weaker models on simpler tasks and stronger models on intermediate tasks, but degrades performance on advanced circuits (-17pp for Opus and Haiku). The mid-tier model (Sonnet) is entirely unaffected by TV at all difficulty levels, suggesting that CoT prompting only impacts performance near a model's capability boundary.

Beyond the benchmark, QCV-Dataset has practical implications for quantum-safe blockchain systems. We include 31 blockchain-relevant circuits covering quantum threats (Shor vs. ECDSA, Grover vs. SHA-256/AES, PoW quantum speedup), post-quantum defenses (Kyber, Dilithium, SPHINCS+, Lamport signatures), and quantum-enhanced infrastructure (QKD networks, random beacons, quantum timestamps, Merkle tree verification). Validation on 11 blockchain circuits achieves 9/11 pass rate, with the 8-qubit consensus protocol passing while 5-qubit and 7-qubit irregular circuits fail—revealing that structural regularity, not qubit count, determines success.

Looking forward, QCV-Dataset's target descriptions and equivalence pairs lay the foundation for a longer-term goal: enabling AI systems to not merely translate but *discover* novel quantum circuits—analogous to how AlphaFold moved from understanding known protein structures to predicting unknown ones. All data and code are publicly available at https://github.com/QuantBlockchain/quantum-circuit-vision.

References

[1] S.-Z. Wong, G.-W. Wan, D. Liu, and X. Wang, "VGV: Verilog Generation using Visual Capabilities of Multi-Modal Large Language Models," in *Proc. IEEE/ACM DAC*, 2024.

[2] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and Opportunities in Conversational Hardware Design," in *Proc. ACM/IEEE MLCAD*, 2023, pp. 1–6.

[3] S. Thakur *et al.*, "VeriGen: A Large Language Model for Verilog Code Generation," 2023.

[4] Y. Fu *et al.*, "GPT4AIGChip: Towards Next-Generation AI Accelerator Design Automation via Large Language Models," in *Proc. ICCAD*, 2023.

[5] M. Liu *et al.*, "ChipNemo: Domain-Adapted LLMs for Chip Design," *arXiv preprint arXiv:2311.00176*, 2023.

[6] Z. He *et al.*, "ChatEDA: A Large Language Model Powered Autonomous Agent for EDA," in *Proc. MLCAD*, 2023.

[7] E. Farhi, J. Goldstone, and S. Gutmann, "A Quantum Approximate Optimization Algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[8] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge University Press, 2010.

[9] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.

[10] Amazon Web Services, "Amazon Braket Developer Guide," https://docs.aws.amazon.com/braket/, 2024.

[11] Qiskit Contributors, "Qiskit: An Open-Source Framework for Quantum Computing," 2023. [Online]. Available: https://qiskit.org/

[12] P. Lu *et al.*, "MathVista: Evaluating Mathematical Reasoning of Foundation Models in Visual Contexts," *arXiv preprint arXiv:2310.02255*, 2023.

[13] A. W. Cross *et al.*, "OpenQASM 3: A Broader and Deeper Quantum Assembly Language," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, 2022.

[14] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in *Proc. NeurIPS*, 2023.

[15] OpenAI, "GPT-4 Technical Report," Tech. Rep., 2023.

[16] A. K. Fedorov, E. O. Kiktenko, and A. I. Lvovsky, "Quantum Computers Put Blockchain Security at Risk," *Nature*, vol. 563, pp. 465–467, 2018.

[17] D. Liu, A. Zhang, and L. Zhang, "Quantum-Safe, Efficient, and AI-Enhanced Blockchains for the Web: A Cooperative Tutorial on Quantum Computing, Blockchain Applications, and Data Standards," in *Companion Proc. ACM Web Conference (WWW Companion '26)*, 2026.

TABLE V
BLOCKCHAIN CIRCUIT VERIFICATION RESULTS (OPUS BV)

Circuit	Application	Qubits	Fidelity	Pass
QRNG	Consensus randomness	4	1.000	✓
BV Oracle	Key extraction analysis	3	1.000	✓
Grover Search	Mining threat model	2	1.000	✓
BB84 QKD	Secure key distribution	4	1.000	✓
Lattice PQC	PQC security analysis	5	0.073	×
QSS (GHZ)	Multi-party secret sharing	4	1.000	✓
Coin Flip	Fair commit-reveal	3	1.000	✓
Oblivious Xfer	Private data exchange	5	1.000	✓
Quantum VRF	Verifiable randomness	6	1.000	✓
QDS	Digital signature	7	0.500	×
Consensus	Validator agreement	8	1.000	✓

APPENDIX

To demonstrate QCV's applicability to quantum-safe blockchain systems, we designed six blockchain-relevant quantum circuits spanning different security functions and complexity levels (Fig. 5), and tested them with Claude Opus 4.6 under BV mode. Results are summarized in Table V.

Nine of eleven circuits were correctly generated (81.8%), including the most complex 8-qubit consensus protocol (256×256 unitary verification) and the 6-qubit VRF circuit requiring manual CRz decomposition. Two circuits failed, each revealing a distinct failure mode.

Passed circuits. (a) QRNG: parallel Hadamard gates for consensus randomness. (b) BV oracle: hidden bitstring extraction for PQC key leakage analysis [16]. (c) Grover: quadratic-speedup attack on proof-of-work mining. (d) BB84: quantum key distribution for secure inter-node communication [17]. (f) GHZ-based quantum secret sharing with phase-encoded shares. (g) Quantum coin flipping: commit-reveal protocol with entanglement and phase rotation. (h) Oblivious transfer: 5-qubit protocol with dual CNOT-Rz-CNOT phase kickback structure. (i) Quantum VRF: 6 qubits with GHZ preparation, cross-register entanglement, and three CRz gates decomposed into CNOT-Rz sequences. (k) Consensus protocol: the largest circuit (8 qubits, 20 gates) with a regular validator-pair structure—H, CNOT pairs, ring-topology cross-links, phase oracle, and final Hadamard—all correctly generated.

Failed circuits. (e) The lattice-inspired PQC circuit (5 qubits, 15 gates, fidelity 0.073) failed because Opus misinterpreted parallel CNOT-Rz-H layers as a sequential cascade. (j) The quantum digital signature circuit (7 qubits, 16 gates, fidelity 0.500) failed because Opus confused the temporal ordering of cross-register CNOTs: it placed CNOT(1,4) before CNOT(3,4) and omitted the parallel CNOT(0,3) step, producing a structurally different circuit.

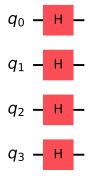
A. Insights: What Determines Success?

The 11-circuit blockchain case study, combined with the 21-circuit main benchmark, reveals a clear pattern about what makes quantum circuits easy or hard for MMLLMs:

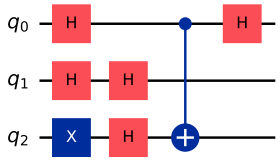
Structural regularity matters more than qubit count. The 8-qubit consensus circuit passed (fidelity 1.0) while the 5-qubit lattice circuit and 7-qubit QDS circuit failed. The consensus circuit has a highly regular structure—repeating H-CNOT-CNOT-Rz-H blocks across four identical validator pairs—which the model can recognize as a pattern. In contrast, the failed circuits have irregular, cross-cutting entanglement structures where gates on different qubits interact in non-repeating ways.

Parallel gate layers are the primary failure mode. Both failed circuits involve gates that execute simultaneously on different qubits but must be serialized in code. The model struggles to determine the correct temporal ordering when multiple gates appear at the same circuit depth. This is consistent with the main benchmark finding that QAOA (which has parallel cost-layer CNOTs) is the hardest circuit for all models.

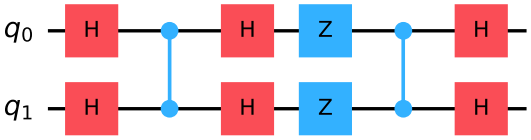
Gate decomposition is not the bottleneck. The 6-qubit VRF circuit required decomposing three CRz gates into CNOT-Rz-CNOT sequences—a non-trivial transformation—yet passed with fidelity 1.0. This suggests that the model has strong knowledge of standard gate decompositions; the challenge lies in visual layout interpretation rather than quantum computing knowledge.



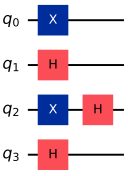
(a) QRNG (4q)



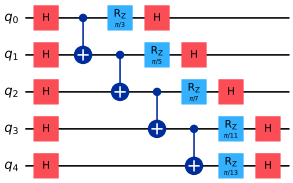
(b) BV Oracle (3q)



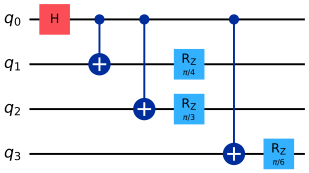
(c) Grover Search (2q)



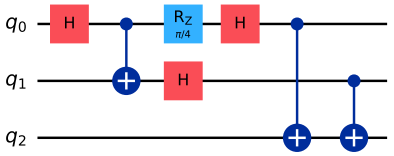
(d) BB84 QKD (4q)



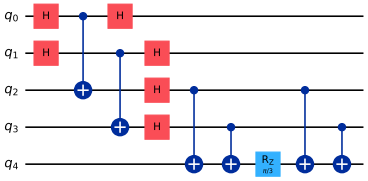
(e) Lattice PQC (5q)



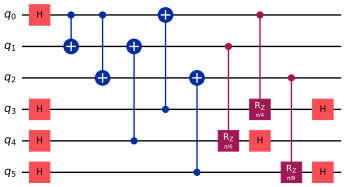
(f) Quantum Secret Sharing (4q)



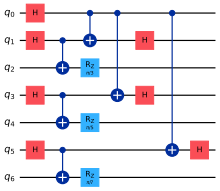
(g) Coin Flip (3q)



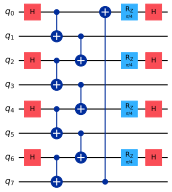
(h) Oblivious Transfer (5q)



(i) Quantum VRF (6q)



(j) Digital Signature (7q)



(k) Consensus Protocol (8q)

Fig. 5. Eleven blockchain-relevant quantum circuits (2–8 qubits) spanning consensus, cryptography, threat modeling, key distribution, secret sharing, fair protocols, private exchange, verifiable randomness, digital signatures, and validator agreement.